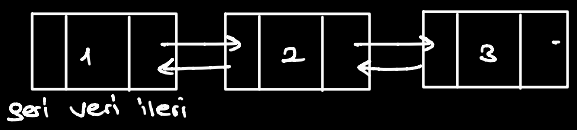
Diğer Linked List Çeşitleri

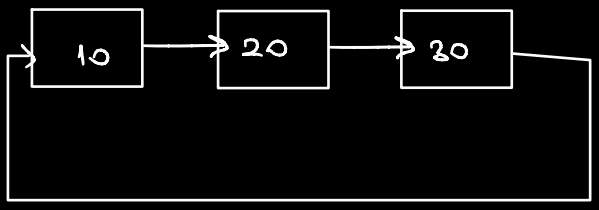
1. Doubly Linked List - Çift yönlü bağlı liste

* Her düğümde üç bölüm olur.




```
Class CiftEleman{
    int veri;
    CiftEleman ileri;
    CiftEleman geri;
    CiftEleman(int veri){
      this.veri=veri;
      this.geri = null;
      this.ileri=null;
   }
```

2. Circular linked list - Dairesel Bağlı Liste




```
void DaireselListe()
   if(bos==null)return;
    Eleman gezici=bos;
    do{
     System.out.println(gezici.veri);
      gezici=gezici.ileri
    }while(gezici!=null);
}
```

III. Bölüm - Özyinemeli Algoritmalar, Bağlı Liste İşlemleri ve Stack (yığın) veri yapısı

Recursive (Öz yineleme)

* Recursive, bir fonksiyonun bir problemi daha küçük alt problemlere ayırarak kendini tekrar çağırmasıdır.

* Recursive fonksiyonlar:

- Base Case (Temel Durum): Fonksiyonun durmasını sağlar
- Recursive Case (Öz yinelemeli Durum): Fonksiyonun kendini çağırması

Örnek:

```
Public Static int faktoryel(int x){
    if(x<=1) return 1;          → Base Case
    return x*faktoryel(x-1);    → Recursive Case
```

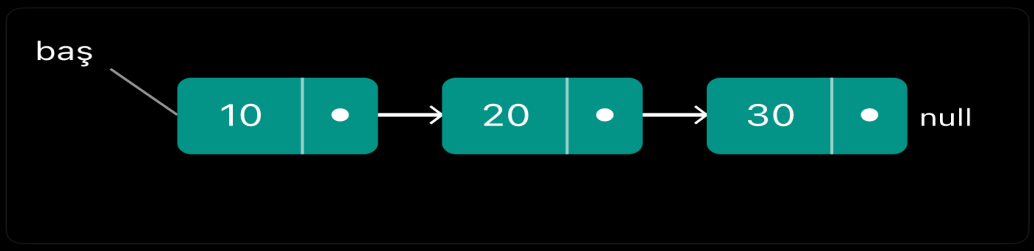
Örneğin $4!$ işlemini hesaplayalım.

$$4! = 4 \times 3 \times 2 \times 1 = 24$$



Recursive Bağlı liste

* Bağlı listedeki her düğüm, bir değer ve sonraki düğüme ait referans içerir.



1. Recursive Gezinme

```
void gez (Eleman aktif){
   if(aktif==null) return
   System.out.println(aktif.sayi);
    gez(aktif.ileri);
 }
```

- Çalışma sırası;

  gez(10) → gez(20) → gez(30) → gez(null) → dur

- Bu kodda yazdırma işlemi çağrıdan önce yapıldığı için elemanları baştan sona yazar.

2. Recursive Sona eleman ekleme

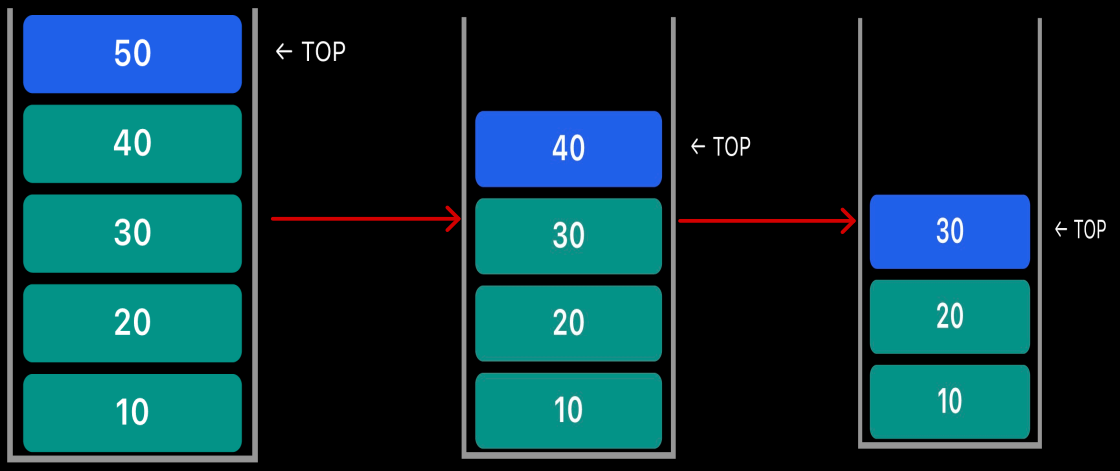
```
Eleman recSonaEkle(Eleman aktif, int sayi){
   if(aktif==null) return new Eleman(sayi);
  aktif.ileri = recSonaEkle(aktif.ileri, sayi);
  retur aktif
```

3. Recursive Sondan silme

```
Eleman recSondanSil(Eleman aktif){
    if(aktif==null) return null;
    if(aktif.ileri==null) return null;
    aktif.ileri = recSondanSil(aktif.ileri);
     return aktif;
  }
```

Stack (Yığın) Veri Yapısı

* Stack, elemanlar LIFO (Last in first Out) kuralına göre işlendiği doğrusal veri yapısıdır.
* Yığına en son eklenen ilk çıkar



Stack işlemleri

push(x): Eleman ekler

pop(): Eleman çıkarır

peek(): üstekı değeri döndürür.

isEmpty(): yığın boş mu

isfull(): yığın dolu mu?

```
public class StackCuston(){
        int[] dizi =new dizi[]
         int top=-1;
        boolean isEmpty(){
           return top==-1;
         }
        boolean isfull(){
           return top== dizi.leght-1;
         }
       void push(int sayi){
          if(isFull()){
             System.out.println("Stack Dolu");
              return;
            }
            dizi[++top]=sayi;
        }
       void pop(){
          if(isEmpty){
             throw new IllegalStatementExepction("Stack bos")
            }
           return dizi[--top];
       }
       void peek(){
        if(isEmpty){
           throw new IllegalStatementExepction("Stack bos")
          }
         return dizi[top];
       }
```

Recursive ve Stack İlişkisi

*Recursive fonksiyonlar calışırken, fonksiyon çağrıları, programın Call Stack yapısında tutulur.

*Her çağrı için bir stack frome oluşur. Bu cerceve genellikle fonksiyonun parametreleri, yerel değişkenleri ve geri dönüş bilgileri olur.

```java
static void yaz(int n) {
    if (n == 0) {
        return;
    }

    System.out.println(n);
    yaz(n - 1);
}
```

`yaz(3)` çağrıldığında:



Her çağrı ayrı bir frame oluşturur.

Çağrılar şu sırayla oluşur:

$$yaz(3) \rightarrow yaz(2) \rightarrow yaz(1) \rightarrow yaz(0)$$

Ardından geri dönüş:

$$yaz(0) \rightarrow yaz(1) \rightarrow yaz(2) \rightarrow yaz(3)$$

Bu örnekte ekrana `3 2 1` yazdırılır; çünkü yazdırma işlemi recursive çağrıdan önce gerçekleşir.

4.1. StackOverflowError

Recursive fonksiyonun temel durumu yoksa veya çağrılar aşırı derinleşirse çağrı yığını kapasitesi aşılabilir.

```java
static void hata() {
    hata();
}
```

Bu kod sonunda `StackOverflowError` oluşturur.

Recursive algoritma tasarlarken temel durumun erişilebilir olması ve her çağrının bu duruma yaklaşması gerekir.